

Computer Networks

Week 4 — Detailed Study Notes

Reliable Delivery: Stop-and-Wait & ABP · TCP Header · Sliding Windows ·
Go-Back-N & Selective Repeat · Socket Buffers · Ordered Delivery

IIT Madras BS — Prof. Ayon Chakraborty

How to read these notes. Every slide concept from the three Week-4 lectures appears here, in slide order, grouped by module (Module 4. n = Lecture n). Box types: **Definition**, **Intuition**, **Formula** (derivations + [units]), **Common pitfall**, **Beyond the slides**, grey **What the slide says**. The end has a formula sheet, numerical problems with full solutions, and MCQs with explanations.

Contents

1	Module 4.1 — Reliability: Stop-and-Wait	3
1.1	How do apps get performance guarantees?	3
1.2	The problem: packet loss	3
1.3	Mechanism 1: Acknowledgements (ACK)	4
1.3.1	ACKs also detect corruption (with a checksum)	4
1.4	Mechanism 2: Retransmission timeout (RTO)	5
1.4.1	How should the RTO be set?	5
1.5	Mechanism 3: Sequence numbers	6
1.5.1	The duplication problem	6
1.5.2	What is the sequence number of the third packet?	7
1.6	Summary: stop-and-wait reliability	8
1.7	ABP — the Alternating Bit Protocol	8
1.7.1	The four ABP cases on the slides	9
2	Module 4.2 — TCP Metadata, Sliding Windows and Go-Back-N	10
2.1	Review: stop-and-wait is slow	11
2.2	Where are sequence and ACK numbers written? The TCP header	12
2.3	Making reliable transmissions efficient: pipelining	13
2.4	Tracking packets in flight: the window	14
2.5	Sliding windows	14
2.5.1	Sender's view	14
2.5.2	The window slides	15
2.5.3	Receiver's view	15
2.5.4	Summary of sliding windows	16
2.6	Making retransmissions efficient	16
2.6.1	Q1: Identifying dropped packets	16
2.6.2	Receiver strategies upon packet loss	17
2.7	Sliding window with Go-Back-N	18
3	Module 4.3 — Selective Repeat, Buffers and Ordered Delivery	19
3.1	Selective repeat with cumulative ACK	19
3.2	Selective repeat with selective ACK (SACK)	20
3.3	Comparing the three reliability schemes	21
3.4	Memory buffers in the transport layer	21
3.4.1	Receiver-side sockets need memory buffers	21
3.4.2	Sender-side sockets need memory buffers	22
3.5	Q2: How much data to keep in flight?	22

3.6	Ordered delivery	23
3.6.1	Reordering packets at the receiver side	23
3.6.2	Why reordering happens, and the fix	23
3.6.3	Interaction between apps and TCP: reassembly	24
3.6.4	Sequence numbers in the app's stream	24
3.6.5	Implications of ordered delivery	25
4	Formula Sheet	26
5	Numerical Problems with Worked Solutions	26
6	Multiple-Choice Questions with Explanations	28

1 Module 4.1 — Reliability: Stop-and-Wait

1.1 How do apps get performance guarantees?

What the slide says (W4 L1, slide 4)

The network core provides no guarantees on packet delivery. **Transport** software on the endpoint oversees implementing guarantees on top of a best-effort network. Three important kinds of guarantees: **reliability**, **ordered delivery**, **resource sharing in the network core** — all provided by the **Transmission Control Protocol (TCP)**.

From the lecture slides: W4 L1, slides 4 and 6 — TCP's three guarantees; the packet-loss problem

How do apps get perf guarantees? Packet loss

- The network core provides no guarantees on packet delivery



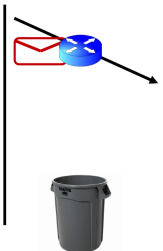
- Transport software on the endpoint oversees implementing guarantees on top of a best-effort network

- Three important kinds of guarantees

- **Reliability**
- Ordered delivery
- Resource sharing in the network core

Transmission Control Protocol (TCP)

Sender Receiver



- How might a sender and receiver ensure that data is delivered reliably (despite some packets being lost)?

- TCP uses three mechanisms

this week

Reliability

every byte arrives

this week

Ordered delivery

in the sent order

later

Resource sharing

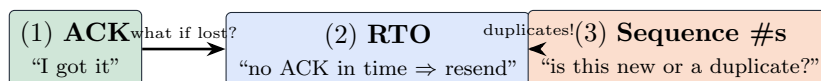
congestion control

TCP, on top of best-effort IP

1.2 The problem: packet loss

What the slide says (W4 L1, slide 6)

How might a sender and receiver ensure that data is delivered reliably (despite some packets being lost)? TCP uses **three mechanisms**.

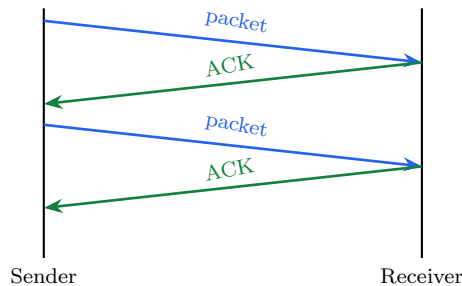


Each mechanism fixes a problem the previous one creates — follow that chain and the whole design makes sense.

1.3 Mechanism 1: Acknowledgements (ACK)

What the slide says (W4 L1, slide 7)

Key idea: the receiver returns an **acknowledgment (ACK)** per packet sent. If the sender receives an ACK, it knows that the receiver got the packet.



1.3.1 ACKs also detect corruption (with a checksum)

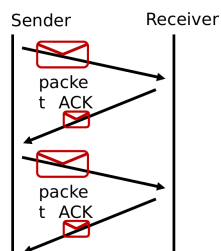
What the slide says (W4 L1, slide 8)

ACKs also work to detect packet corruption on the way to the receiver. One possibility: the receiver sends a **negative acknowledgment (NAK)** if it receives a corrupted packet. Q: How to detect a corrupted packet? One method: **checksum!** **TCP only uses positive ACKs.**

From the lecture slides: W4 L1, slides 7–8 — ACKs; NAK for corruption (TCP uses only positive ACKs)

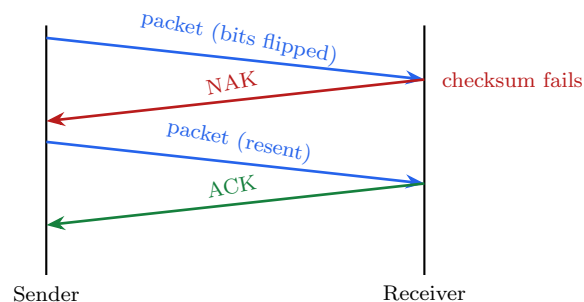
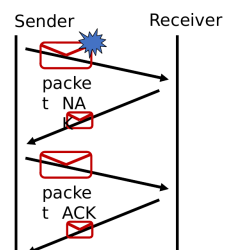
Coping with packet loss: (1) ACK

- Key idea: Receiver returns an **acknowledgment (ACK)** per packet sent
- If sender receives an ACK, it knows that the receiver got the packet.



Coping with packet corruption: (1) ACK

- ACKs also work to detect packet corruption on the way to the receiver
 - One possibility: A receiver could send a negative acknowledgment, or a **NAK**, if it receives a corrupted packet
 - Q: How to detect corrupted packet?
 - One method: **Checksum!**
- TCP only uses positive ACKs.



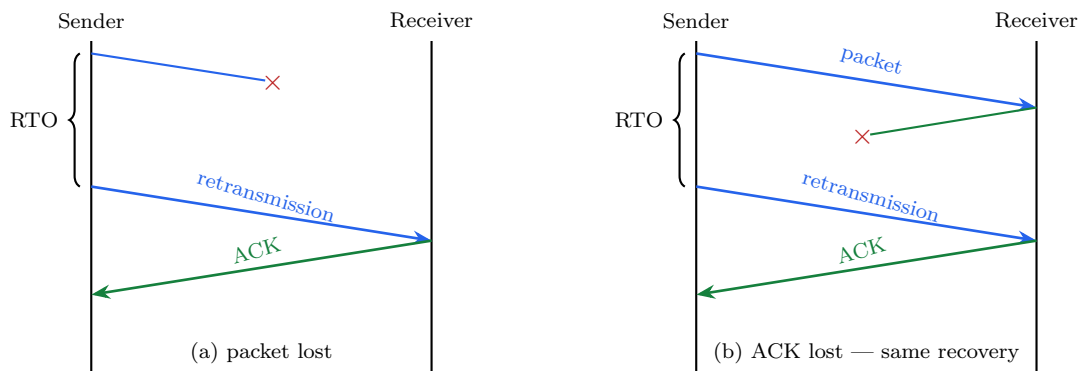
Intuition: Why TCP doesn't bother with NAKs

A NAK can itself be lost or corrupted, so the sender still needs a timeout to be safe. And a timeout already handles corruption if the receiver simply *discards* a bad packet and says nothing: from the sender's view, a corrupted packet looks exactly like a lost one. So positive ACKs + timeout cover both cases with one mechanism. (NAKs can speed recovery; TCP gets a similar effect from duplicate ACKs, Module 4.3.)

1.4 Mechanism 2: Retransmission timeout (RTO)

What the slide says (W4 L1, slide 9)

What if a packet is dropped? Key idea: wait for a duration of time (called **retransmission timeout** or RTO) before **re-sending** the packet. In TCP, **the onus is on the sender** to retransmit lost data when ACKs are not received. Note that retransmission works also if **ACKs are lost or delayed**.



1.4.1 How should the RTO be set?

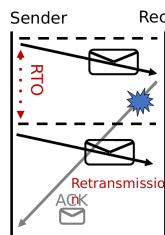
What the slide says (W4 L1, slide 10)

A good RTO must **predict the round-trip time (RTT)** between sender and receiver. RTT: the time to send a single packet and receive a (corresponding) single ACK at the sender. Intuition: if an ACK hasn't returned, and our (best estimate of) RTT has elapsed, the packet was likely dropped. RTT can be measured directly at the sender — no receiver involvement needed.

From the lecture slides: W4 L1, slides 9–10 — retransmission timeout and how to set it

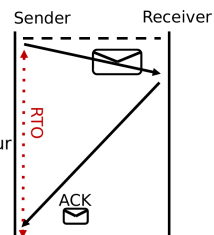
Coping with packet loss: (2) RTO

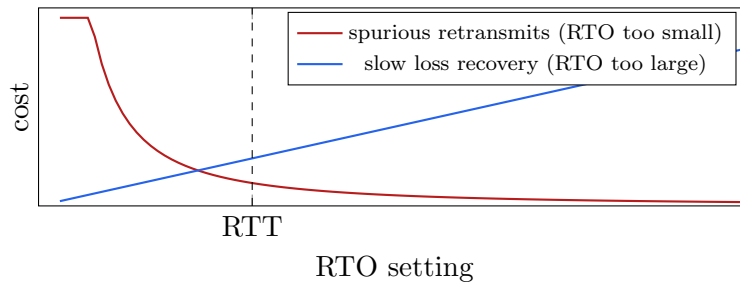
- What if a packet is dropped?
- Key idea: Wait for a duration of time (called **retransmission timeout** or RTO) before **re-sending** the packet
- In TCP, **the onus is on the sender** to retransmit lost data when ACKs are not received
- Note that retransmission works also if **ACKs are lost or delayed**



How should the RTO be set?

- A good RTO must **predict the round-trip time (RTT)** between the sender and receiver
 - RTT: the time to send a single packet and receive a (corresponding) single ACK at the sender
- Intuition: If an ACK hasn't returned, and our (best estimate of) RTT has elapsed, the packet was likely dropped.
- RTT can be measured directly at the sender. No receiver involvement needed.





Intuition

Too small an RTO fires before the ACK could possibly return $\Rightarrow$ needless duplicates that waste bandwidth (and add to congestion). **Too large** an RTO means a genuinely lost packet sits unrecovered for ages. The sweet spot is “a bit more than the RTT” — but RTT varies with queuing, so it must be *estimated* continuously.

Beyond the slides: How TCP actually estimates RTO (RFC 6298)

Each time an ACK returns, the sender measures **SampleRTT**. Then

$$\begin{aligned} \text{EstimatedRTT} &\leftarrow (1 - \alpha) \text{EstimatedRTT} + \alpha \text{SampleRTT} & \alpha &= 1/8 \\ \text{DevRTT} &\leftarrow (1 - \beta) \text{DevRTT} + \beta |\text{SampleRTT} - \text{EstimatedRTT}| & \beta &= 1/4 \\ \text{RTO} &= \text{EstimatedRTT} + 4 \text{DevRTT} \end{aligned}$$

An exponentially weighted moving average smooths noise; the $4\times$ deviation margin keeps spurious timeouts rare when RTT is jittery. Also, on each timeout the RTO is doubled (exponential backoff), and samples from retransmitted packets are ignored (Karn’s algorithm) because it’s ambiguous which transmission the ACK belongs to.

1.5 Mechanism 3: Sequence numbers

1.5.1 The duplication problem

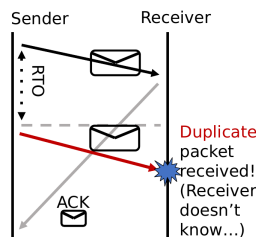
What the slide says (W4 L1, slide 11)

If ACKs are delayed beyond the RTO, the sender may retransmit the *same* data. The receiver wouldn’t know that it just received duplicate data from this retransmitted packet. Add some identification to each packet to help distinguish between adjacent transmissions: this is the **sequence number**.

From the lecture slides: W4 L1, slides 11–12 — duplication from a delayed ACK; sequence numbers disambiguate

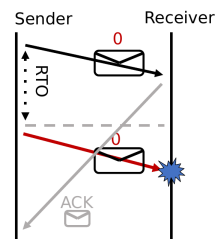
Coping with packet duplication

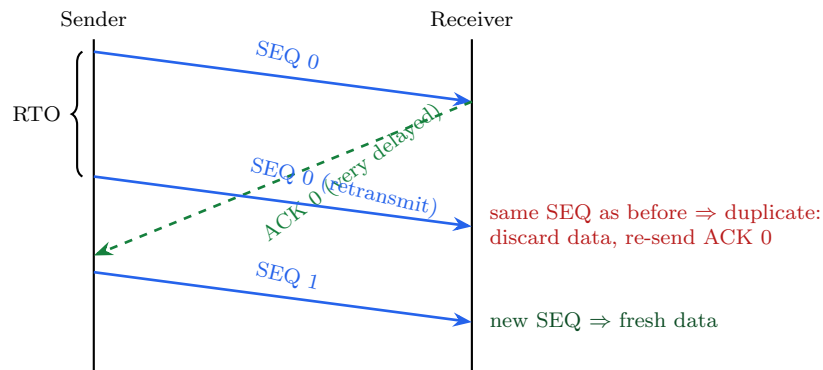
- If ACKs delayed beyond the RTO, sender may retransmit the **same** data
 - Receiver wouldn’t know that it just received duplicate data from this retransmitted packet
- Add some identification to each packet to help distinguish between adjacent transmissions
 - This is known as the **sequence number**



Coping with packet loss: (3) Sequence #s

- A bad scenario: Suppose an ACK was delayed beyond the RTO; sender ended up retransmitting the packet.
- At the receiver: **sequence number helps disambiguate a fresh transmission from a retransmission**
 - Sequence number same as earlier: retransmission
 - Fresh sequence number: fresh data





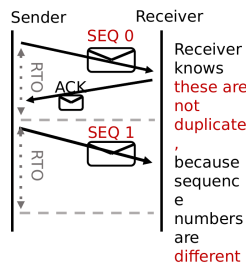
What the slide says (W4 L1, slides 12–14)

Bad scenario: an ACK was delayed beyond the RTO; the sender retransmitted. At the receiver, the sequence number disambiguates: same as earlier $\Rightarrow$ retransmission; fresh $\Rightarrow$ fresh data. **Good scenario:** packet received and ACK returned within RTO; successive packets carry different sequence numbers (SEQ 0, SEQ 1), so the receiver knows these are not duplicates. Further, the receiver tells the sender which packet was ACKed using an **ACK sequence number**.

From the lecture slides: W4 L1, slides 13–14 — the good scenario; ACK sequence numbers

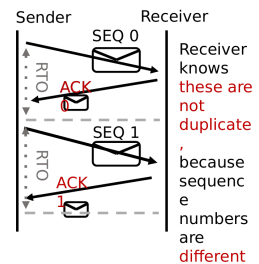
Coping with packet loss: (3)
Sequence #s

- A good scenario: packet successfully received and ACK returned within RTO
- Sequence numbers of successively transmitted packets are different



Coping with packet loss: (3)
Sequence #s

- A good scenario: packet successfully received and ACK returned within RTO
- Sequence numbers of successively transmitted packets are different
- Further, the receiver informs the sender which packet was ACK'ed using an **ACK sequence number**



Intuition: Why ACKs need numbers too

Without ACK numbers, a late “ACK” for an old packet could be mistaken for the ACK of the current one — the sender would think the current packet arrived when it didn’t, and the data would be silently lost. Numbering ACKs (“ACK 0”, “ACK 1”) removes this ambiguity on the sender’s side just as sequence numbers do on the receiver’s side.

1.5.2 What is the sequence number of the third packet?

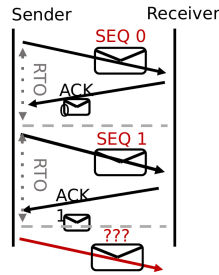
What the slide says (W4 L1, slide 15)

Goal: avoid ambiguity on which packet was received/ACKed from both the sender's and receiver's perspective. One possibility: keep incrementing the seq #: 2, 3, Alternative: since seq # 0 was successfully ACKed earlier, it is OK to **reuse** seq #0 for the next transmission. Seq #s can be reused if enough time has elapsed.

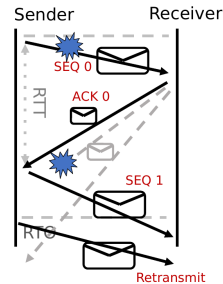
From the lecture slides: W4 L1, slides 15–16 — what is the third seq #?; stop-and-wait summary

Q: What is the seq# of third packet? Summary: Stop-and-Wait Reliability

- Goal: Avoid ambiguity on which packet was received/ACK'ed from both the sender and receiver's perspective
- One possibility: keep incrementing the seq #: 2, 3, ...
- Alternative: since seq # 0 was successfully ACK'ed earlier, it is OK to reuse seq #0 for next transmission.
 - Seq #s reused if enough time elapsed



- Sender sends a single packet, then waits for an ACK to know the packet was successfully received. Then the sender transmits the next packet.
- If ACK is not received until a timeout (RTO), sender **retransmits** the packet
- Disambiguate duplicate vs. fresh packets using sequence numbers that change on “adjacent” packets



Intuition

In stop-and-wait, there is only ever one packet outstanding. The receiver only needs to tell “the packet I’m expecting” from “the one I just accepted”. Two values suffice. Once SEQ 0 has been ACKed and SEQ 1 sent, no old SEQ 0 can still be floating around (assuming the network doesn’t hold packets for ages), so 0 can be reused.

1.6 Summary: stop-and-wait reliability

Summary: Stop-and-wait (slide)

The sender sends a **single** packet, then waits for an ACK to know it was successfully received; then transmits the next packet. If the ACK is not received before a timeout (RTO), the sender **retransmits**. Duplicate vs fresh packets are disambiguated using sequence numbers that change on “adjacent” packets.

1.7 ABP — the Alternating Bit Protocol

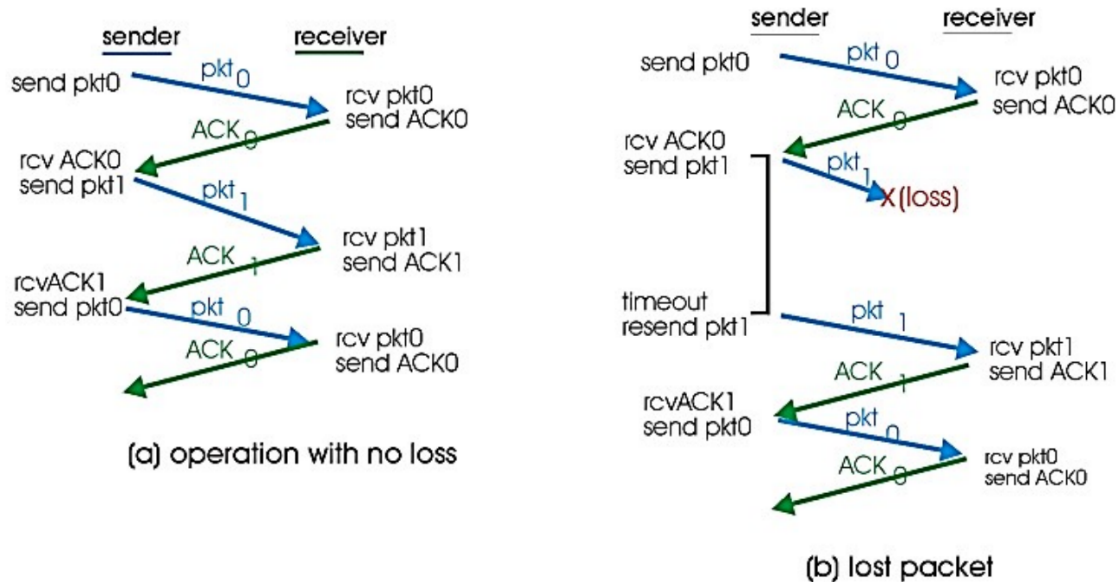
Definition: Alternating Bit Protocol

Stop-and-wait with a **1-bit sequence number** that alternates 0, 1, 0, 1, The receiver ACKs with the bit it received; it delivers a packet only if its bit is the one it expects, otherwise it discards it as a duplicate and re-ACKs.

1.7.1 The four ABP cases on the slides

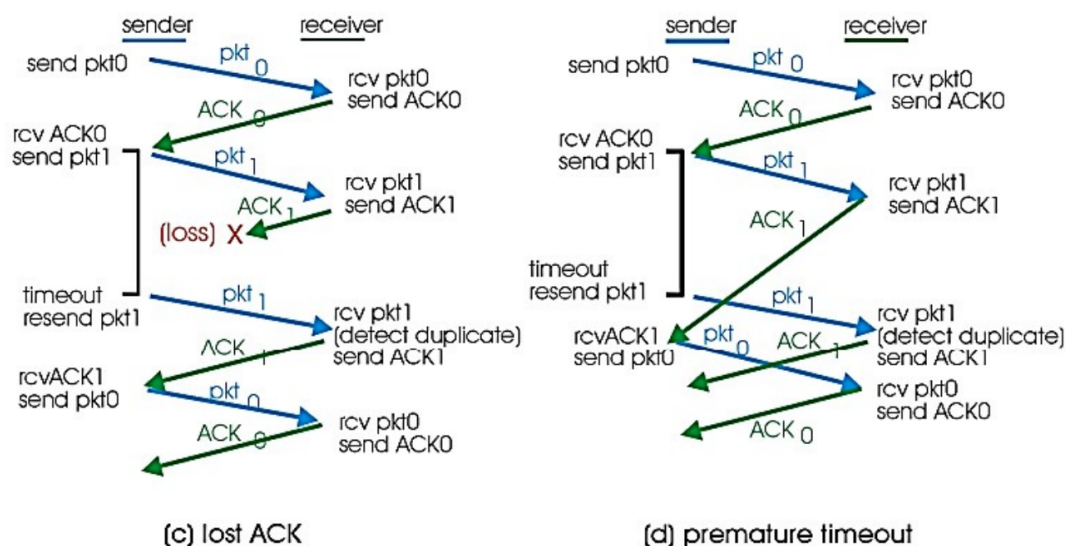
From the lecture slides: W4 L1, slide 17 — (a) operation with no loss, (b) lost packet

ABP – Alternating Bit Protocol



From the lecture slides: W4 L1, slide 18 — (c) lost ACK, (d) premature timeout

ABP – Alternating Bit Protocol

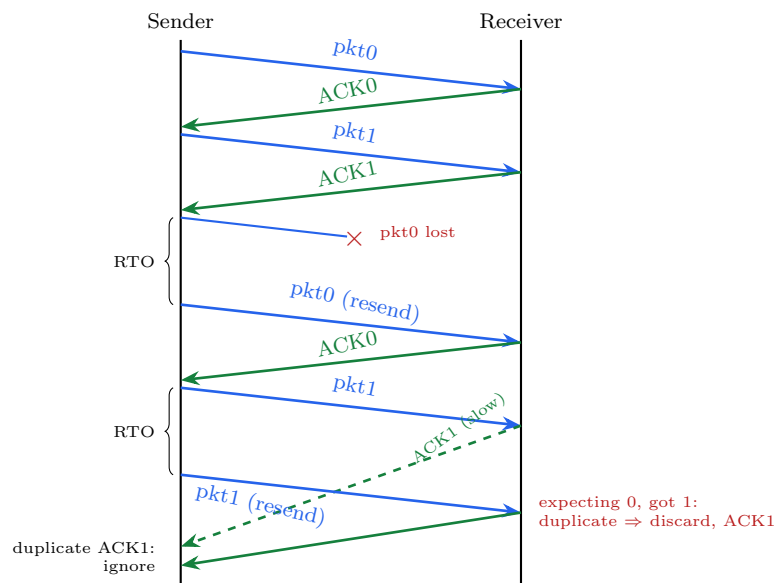


1. **(a) No loss.** `send pkt0` → `rcv pkt0`, `send ACK0` → `rcv ACK0`, `send pkt1` → `rcv pkt1`, `send ACK1` → `rcv ACK1`, `send pkt0` ... The bit alternates on every new packet.
2. **(b) Lost packet.** `pkt1` is lost. The receiver never sees it, so sends nothing. The sender's timer expires: `timeout`, `resend pkt1`. The receiver gets `pkt1` (the bit it expects), delivers it, sends `ACK1`. Normal operation resumes with `pkt0`.

3. (c) **Lost ACK.** pkt1 arrives and is delivered; ACK1 is lost. The sender times out and resends pkt1. The receiver now expects bit 0, sees bit 1 $\Rightarrow$ **detect duplicate**: discard the data but *still send ACK1* (otherwise the sender would retransmit forever). The sender gets ACK1 and moves on.
4. (d) **Premature timeout.** Nothing is lost, but ACK1 is slow, so the sender's timer fires and it resends pkt1. Then two things arrive at the sender: the original ACK1 (it sends pkt0 next) and, later, the ACK1 for the duplicate — which the sender must **ignore** (it is waiting for ACK0 now). At the receiver the duplicate pkt1 is detected and re-ACKed as in (c).

Intuition: What the four cases prove

In every case the receiver delivers each packet *exactly once*, in order: loss is fixed by the timeout, duplicates are caught by the bit, and stale ACKs are ignored because they carry the wrong bit. Case (d) is the reason ACKs must carry the sequence bit: an unnumbered duplicate ACK1 would be mistaken for an ACK of the new pkt0.



Beyond the slides: ABP state machines (compact)

Sender: state = bit b . Send $\text{pkt}(b)$, start timer. On $\text{ACK}(b)$: stop timer, $b \leftarrow 1 - b$, send next. On $\text{ACK}(1 - b)$ or corrupted ACK: ignore. On timeout: resend $\text{pkt}(b)$.

Receiver: state = expected bit e . On $\text{pkt}(e)$ intact: deliver, send $\text{ACK}(e)$, $e \leftarrow 1 - e$. On $\text{pkt}(1 - e)$ or corrupted: send $\text{ACK}(1 - e)$ (i.e. re-ACK the last good one), don't deliver.

Common pitfall: ABP's hidden assumption

ABP is correct only if the channel does **not reorder** packets (FIFO). If a very old *pkt0* could arrive after the current *pkt0* has been accepted, a 1-bit number can't tell them apart. That's why the Internet (which can reorder) uses large sequence numbers and TCP bounds how long a packet may live.

2 Module 4.2 — TCP Metadata, Sliding Windows and Go-Back-N

2.1 Review: stop-and-wait is slow

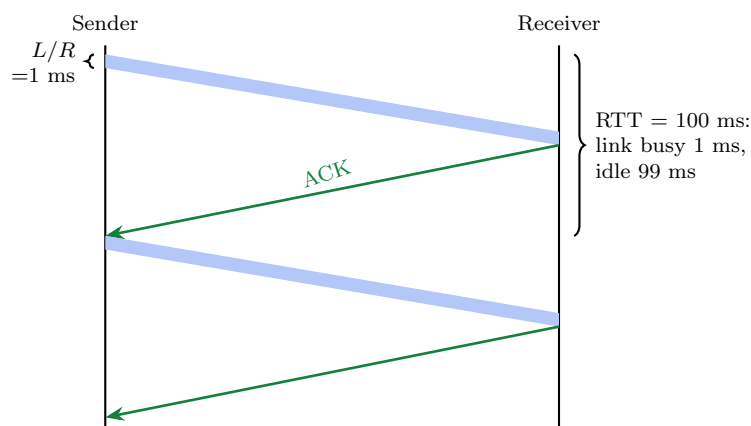
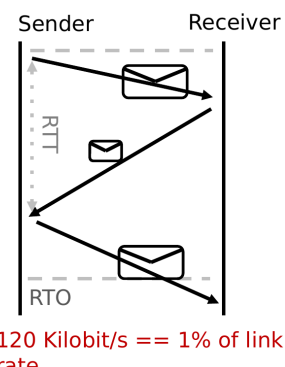
What the slide says (W4 L2, slide 3)

Stop and wait: sender waits for an ACK/RTO before sending another packet. Suppose no packets are dropped; $RTT = RTO = 100$ ms; packet size 12 Kbit ($1K = 10^3$); link rate 12 Mbit/s ($1M = 10^6$). Rate of data transmission? One packet per $RTT = 12 \text{ Kbit}/100 \text{ ms} = 120 \text{ Kbit/s} = \mathbf{1\% \text{ of link rate}}$.

From the lecture slides: W4 L2, slide 3 — stop-and-wait uses 1% of the link

Review: Stop-and-Wait Reliability

- Stop and wait: sender waits for an ACK/RTO before sending another packet
- Suppose no packets are dropped
 - $RTT = RTO = 100$ milliseconds
 - Packet size: 12 Kbit ($1K = 10^3$)
 - Link rate: 12 Mbit/s ($1M = 10^6$)
- Rate of data transmission?
 - one packet per $RTT = 12 \text{ Kbits} / 100 \text{ ms} = 120 \text{ Kbit/s}$



Formula: Stop-and-wait throughput and utilisation

One packet of L bits is delivered per cycle of length $RTT + L/R$:

$$\text{Throughput} = \frac{L}{RTT + L/R} \approx \frac{L}{RTT} \left[\frac{\text{bits}}{\text{s}} \right], \quad U_{\text{sender}} = \frac{L/R}{RTT + L/R} \quad [\text{fraction}]$$

Slide numbers: $L/R = 12 \times 10^3 / 12 \times 10^6 = 1 \text{ ms}$. $U = 1/(100+1) \approx 1\%$ (the slide approximates the cycle as $RTT = 100 \text{ ms}$: $12 \text{ Kbit}/0.1 \text{ s} = 120 \text{ Kbit/s}$).

Intuition

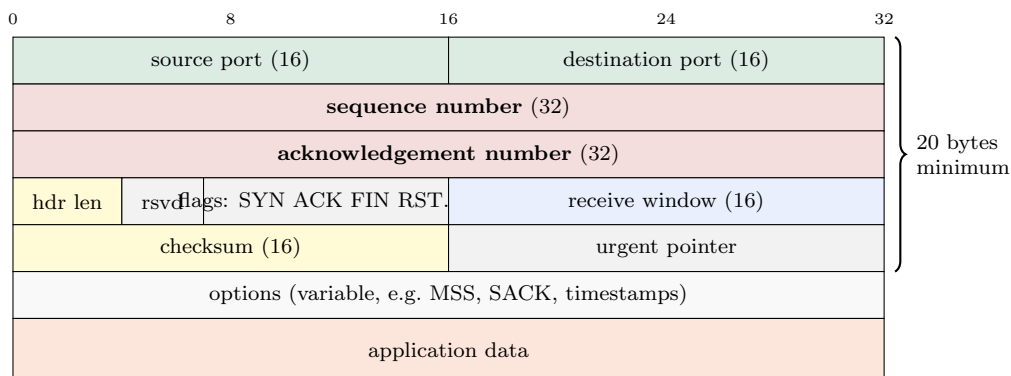
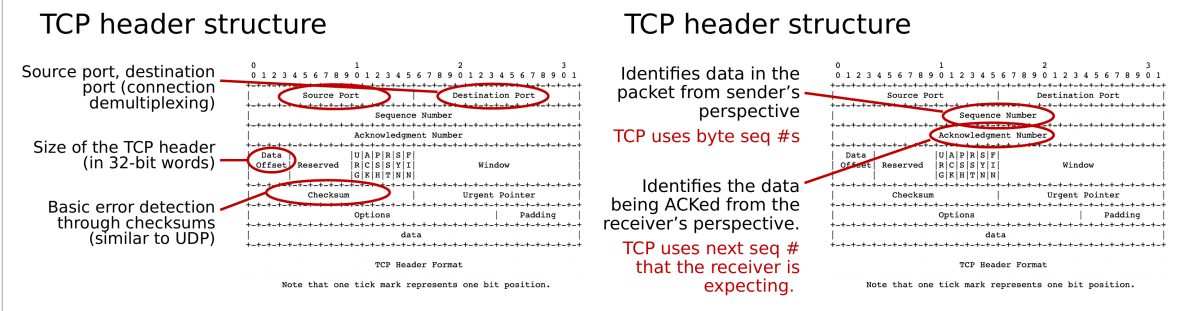
The link can move 12 Mbit/s, but stop-and-wait keeps just one 12-Kbit packet “in the pipe” per round trip. The round-trip pipe holds $R \times RTT = 1.2 \text{ Mbit}$ — 100 packets’ worth — so we’re filling 1% of it. Faster links make this *worse*: at 1 Gbps, utilisation would be 0.012%.

2.2 Where are sequence and ACK numbers written? The TCP header

What the slide says (W4 L2, slides 4–6)

Naturally, in the packet header! TCP header: source port, destination port (connection demultiplexing); size of the TCP header (in 32-bit words); basic error detection through checksums (similar to UDP). **Sequence number:** identifies data in the packet from the sender's perspective — TCP uses **byte seq #s**. **ACK number:** identifies the data being ACKed from the receiver's perspective — TCP uses the **next seq # that the receiver is expecting**.

From the lecture slides: W4 L2, slides 5–6 — the TCP header (RFC 793 figure) with the key fields circled

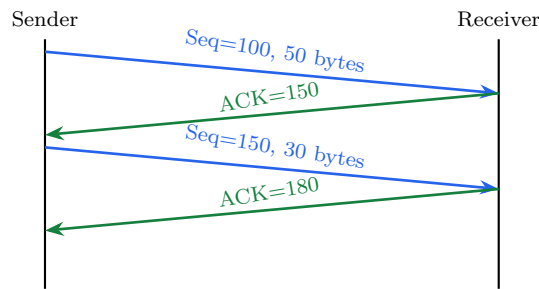


Field	Role
Source / destination port	connection demultiplexing (with the IPs: the 4-tuple)
Sequence number	byte number of the <i>first</i> data byte in this segment
ACK number	next byte the receiver expects (cumulative: all earlier bytes received)
Header length	in 32-bit words: 5 (20 B) to 15 (60 B) — needed because options vary
Flags	SYN/FIN (open/close), ACK (ACK field valid), RST (abort) ...
Receive window	how many more bytes the receiver can buffer (flow control; Q2 below)
Checksum	same Internet checksum as UDP (with pseudo-header)

Formula: TCP byte sequence numbers

If a segment has sequence number x and carries n data bytes, it holds bytes $x, x+1, \dots, x+n-1$, and the next segment's sequence number is $x+n$. On receiving it (with everything before x already received), the receiver sends

$$\text{ACK number} = x + n.$$



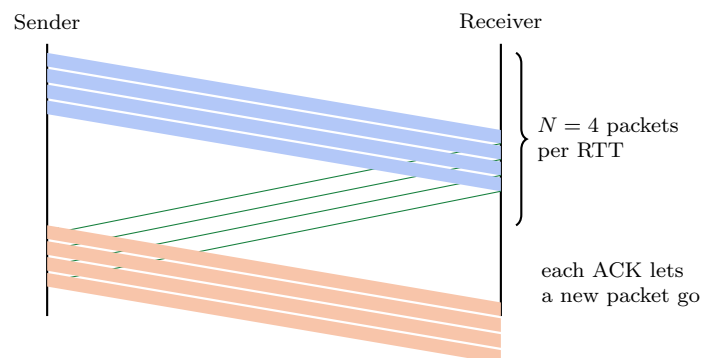
Common pitfall: Off-by-one

The ACK number is the *next expected* byte, not the last byte received. Having bytes 100–149, the receiver sends ACK 150.

2.3 Making reliable transmissions efficient: pipelining

What the slide says (W4 L2, slide 8)

Terminology: **unACKed data / packets in flight** — data that has been sent, but not known (by the sender) to be received. With just one packet in flight, the data rate is limited by the packet delay (RTT) rather than the available bandwidth (link rate). Larger the delay, slower the data rate, regardless of link rate. Idea: **keep many packets in flight!** More packets in flight improves throughput. Such protocols implement **pipelined reliability**.



What the slide says (W4 L2, slide 9)

Why does pipelined reliability help? Suppose the sender has multiple in-flight (yet-to-be-acknowledged) packets. New packets are transmitted concurrently with in-flight packets. Packets and ACKs (of prior packets) are concurrently transmitted $\Rightarrow$ more data and ACKs transmitted within the same duration.

From the lecture slides: W4 L2, slides 8–10 — pipelining; (a) stop-and-wait vs (b) pipelined across the US; tracking packets in flight

Making reliable transmissions efficient

- Terminology: **unACKed data / packets in flight**
 - Data that has been sent, but not known (by the sender) to be received
- With just one packet in flight, the data rate is limited by the packet delay (RTT) rather than available bandwidth (link rate)
 - Larger the delay, slower the data rate, regardless of link rate
- Idea: **Keep many packets in flight!**
 - More packets in flight improves throughput
- We say such protocols implement **pipelined reliability**

Why does pipelined reliability help? Tracking packets in flight

Suppose sender has multiple, in-flight (yet-to-be-acknowledged) packets
New packets transmitted *concurrently* with in-flight packets
Packets and ACKs (of prior packets) are concurrently transmitted

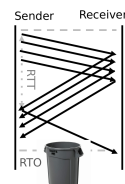
→ More data in flight



- If there are N packets in flight, throughput improves by N times relative to stop-and-wait.
- Stop and wait: send 1 packet per RTT
- Pipelined: send N packets per RTT

We term the in-flight data the **window**

We term the amount of in-flight data the **window size**



The US-map figure (Kurose & Ross) makes the point visually: in (a) one data packet crosses the country and the link sits idle while its ACK comes back; in (b) a stream of data packets and a stream of ACKs are in flight at the same time.

2.4 Tracking packets in flight: the window

Formula: Pipelined throughput

If there are N packets in flight, throughput improves by N times relative to stop-and-wait:

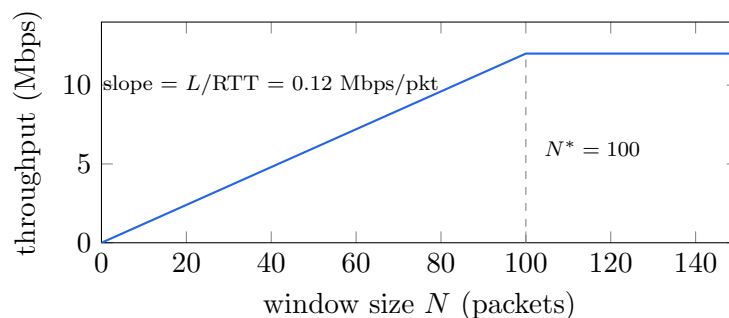
$$\text{Throughput} = \min\left(\frac{NL}{\text{RTT}}, R\right) \quad \left[\frac{\text{bits}}{\text{s}}\right]$$

Derivation. In one RTT the sender can release its whole window (NL bits) before the first ACK returns; each ACK then releases one more packet, so the pattern repeats every RTT. It can't exceed the link rate R . The window that exactly fills the pipe:

$$N^*L = R \times \text{RTT} \quad \Rightarrow \quad N^* = \frac{R \times \text{RTT}}{L} \quad [\text{packets}]$$

Definition: Window, window size

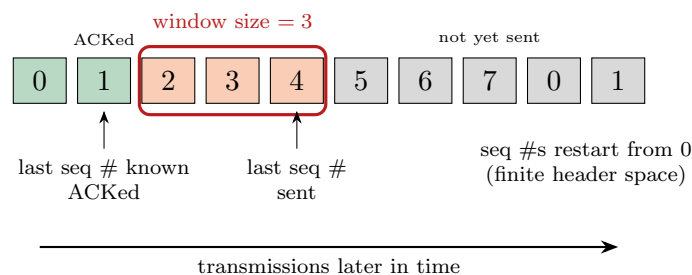
We term the in-flight data the **window** (the sequence numbers of in-flight data); the amount of in-flight (unACKed) data is the **window size**.



(Using the slide's numbers: $L = 12 \text{ Kbit}$, $\text{RTT} = 100 \text{ ms}$, $R = 12 \text{ Mbps}$.)

2.5 Sliding windows

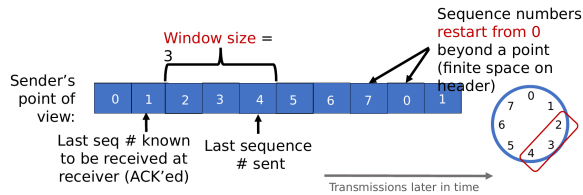
2.5.1 Sender's view



From the lecture slides: W4 L2, slides 12 and 14 — the window, and the sequence-number wheel

Window

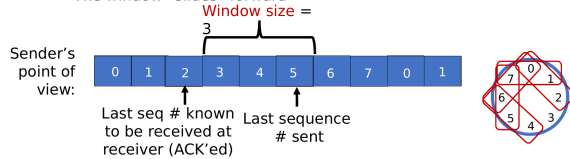
- Window: Sequence numbers of in-flight data
- Window size: The amount of in-flight data (unACKed)



Sliding window (sender side)

- Suppose sequence number 2 is acknowledged by the receiver

- Sender can transmit sequence # 5
- The window “slides” forward



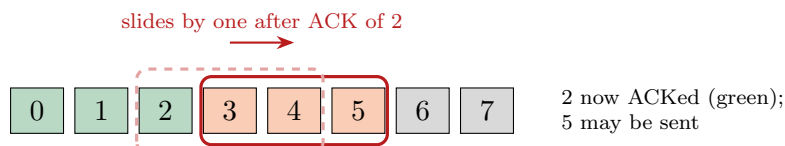
Intuition: The sequence-number wheel

The circle on the slides (0–7 around a clock) is the sequence-number space: with 3 bits, numbers run 0, 1, ..., 7 and then wrap to 0. The window is an arc on this wheel (the red box covering 2, 3, 4, then 3, 4, 5 after the slide). As ACKs arrive the arc rotates around the wheel. The overlapping boxes on slide 14 show successive window positions: the same numbers are reused lap after lap, which is safe only if the window is small enough that an old lap can't be confused with the current one (Module 4.3's 2^{k-1} rule for Selective Repeat).

2.5.2 The window slides

What the slide says (W4 L2, slides 13–14)

Suppose sequence number 2 is acknowledged by the receiver. The sender can transmit sequence # 5. The window “slides” forward.



2.5.3 Receiver's view

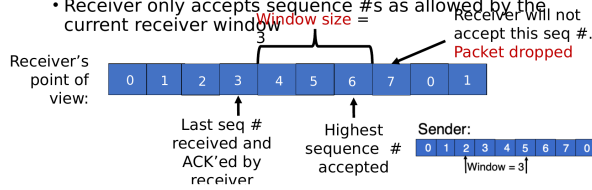
What the slide says (W4 L2, slide 15)

The window of in-flight packets can look different between sender and receiver: the **receiver has more recent info** of in-flight packets. The receiver only accepts sequence #s as allowed by the current receiver window. (Last seq # received and ACKed by the receiver; highest sequence # accepted.) A seq # beyond the window: “Receiver will not accept this seq #. Packet dropped.”

From the lecture slides: W4 L2, slides 15–16 — the receiver's window; summary of sliding windows

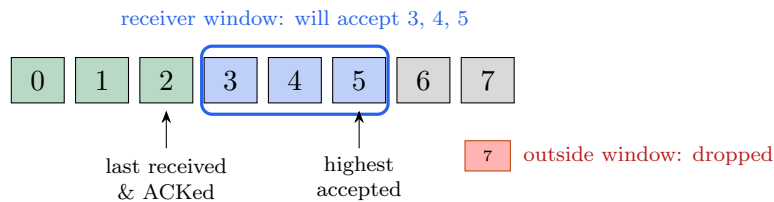
Sliding window (receiver side)

- Window of in-flight packets can look different between sender and the receiver: receiver has more recent info of in-flight
- Receiver only accepts sequence #s as allowed by the current receiver window



Summary of sliding windows

- Sender and receiver can keep several packets of in-flight data
 - Book-keep the sequence numbers using the window
- Windows slide forward as packets are ACKed (at receiver) and ACKs are received (at sender)
- Common case: Improve throughput by sending and ACKing more packets in the same duration
- Key challenge: how should the sender and receiver collaboratively track the packets that must be retransmitted?



Intuition: Why the two windows differ

The receiver sees packets arrive before the sender sees their ACKs. E.g. the receiver has already got 2 and moved its window to 3–5, while the ACK for 2 is still travelling back, so the sender's window is still 2–4. The two views converge one RTT/2 later.

2.5.4 Summary of sliding windows

Summary: Sliding windows (slide)

Sender and receiver can keep several packets of in-flight data. Book-keep the sequence numbers using the window. Windows slide forward as packets are ACKed (at the receiver) and ACKs are received (at the sender). Common case: improve throughput by sending and ACKing more packets in the same duration. Key challenge: how should the sender and receiver collaboratively tackle losses?

2.6 Making retransmissions efficient

What the slide says (W4 L2, slide 18)

Pipelined reliability: N packets in flight $\Rightarrow N \times$ throughput. **Q1:** how should the sender efficiently identify which packets were dropped and (hence) retransmit them? **Q2:** how much data to keep in flight (i.e. what is N ?) to reduce drops/retransmits?

2.6.1 Q1: Identifying dropped packets

What the slide says (W4 L2, slide 20)

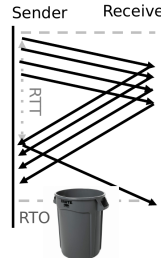
Suppose 4 packets were sent, but one was dropped. How does the sender know which one(s)? Recall: the receiver writes sequence numbers on the ACK; the sender infers which bytes were received successfully using the ACK #s. **Q1.1:** Should receivers ACK subsequent packets upon detecting data loss? **Q1.2:** If so, what sequence number should the receiver put on the

ACK?

From the lecture slides: W4 L2, slides 18 and 20 — pipelined reliability; which packet was dropped?

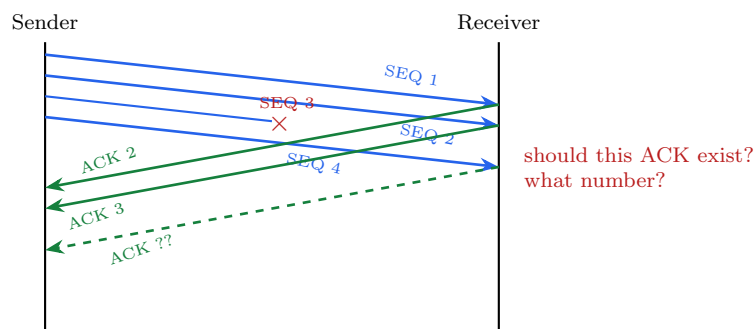
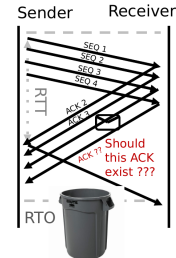
Pipelined Reliability

- If there are N packets in flight, throughput improves by N times relative to stop-and-wait.
 - Stop and wait: send 1 packet per RTT
 - Pipelined: send N packets per RTT
- Q1: how should sender efficiently identify which pkts were dropped and (hence) retransmitted?
- Q2: how much data to keep in flight (i.e., what is N ?) to reduce drops/retransmits?

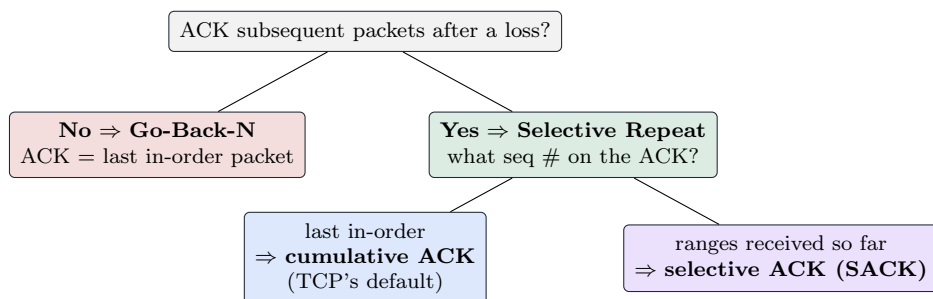


Q1: Identifying dropped packets

- Suppose 4 packets were sent, but one was dropped. How does sender know which one(s) were dropped?
- Recall: Receiver writes **sequence numbers** on the ACK
 - Sender infers which bytes were received successfully using the ACK #'s
- Q1.1: Should receivers ACK subsequent packets upon detecting data loss?
- Q1.2: If so, what sequence number should receiver put on the ACK?

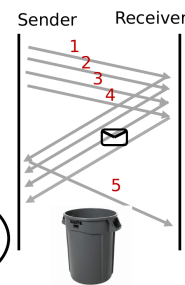
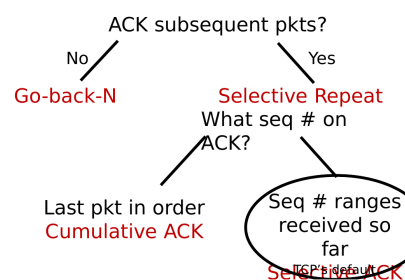


2.6.2 Receiver strategies upon packet loss



From the lecture slides: W4 L2, slides 21–22 — receiver strategies; Go-Back-N rule

Receiver strategies upon packet loss



Sliding Window with Go Back N

- When the receiver notices a missing or erroneous frame:
- It simply discards all frames with greater sequence numbers
 - The receiver will send no ACK
- The sender will eventually time out and retransmit all the frames in its sending window

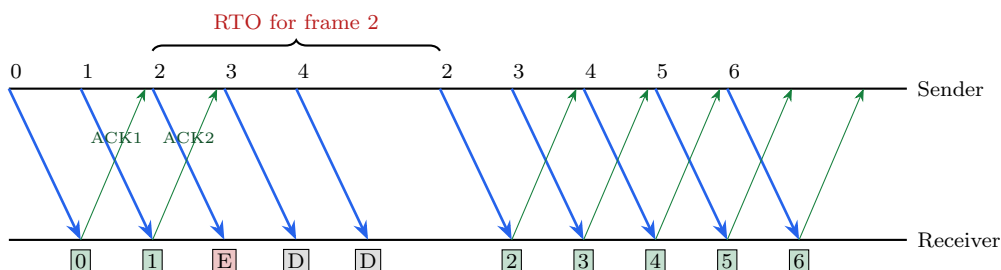
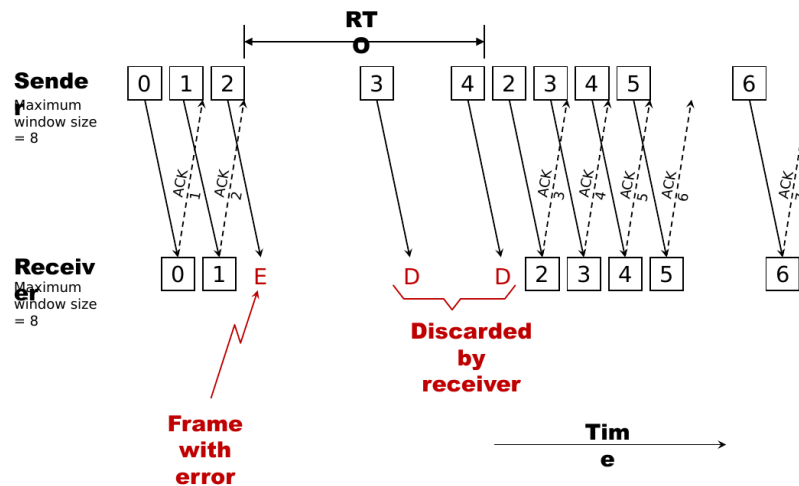
2.7 Sliding window with Go-Back-N

What the slide says (W4 L2, slide 22)

When the receiver notices a missing or erroneous frame: it simply **discards all frames with greater sequence numbers**; the receiver will send **no ACK** (for them). The sender will eventually **time out** and **retransmit all the frames in its sending window**.

From the lecture slides: W4 L2, slide 23 — the Go-Back-N trace (window 8)

Go back N



E = error, D = discarded by receiver (arrived fine, but after the hole)

after the RTO the sender goes back to 2 and resends 2, 3, 4, ...

Intuition

The Go-Back-N receiver is simple: it only ever accepts the next in-order frame, so it needs **no buffer** for out-of-order frames (receiver window = 1). The price is paid by the network: after one loss, every frame already sent after it (up to $N - 1$ of them) crosses the network again.

What the slide says (W4 L2, slide 24)

Go Back N can recover from erroneous or missing frames. **But it is wasteful.** If there are errors, the sender will spend time and network bandwidth retransmitting data the receiver has already seen.

Formula: Cost of one loss under Go-Back-N

With window N and one lost frame in a full window, up to N frames are retransmitted (the lost one + up to $N - 1$ discarded successors). Under Selective Repeat, only 1 is.

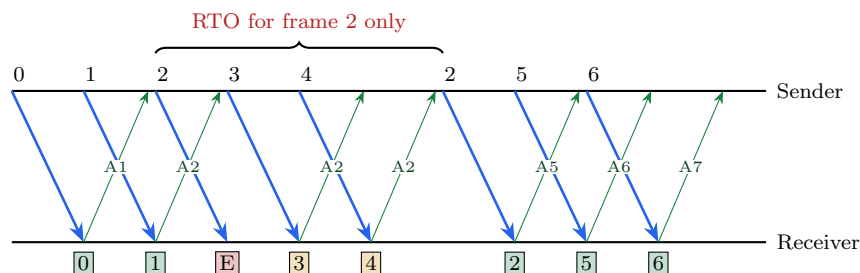
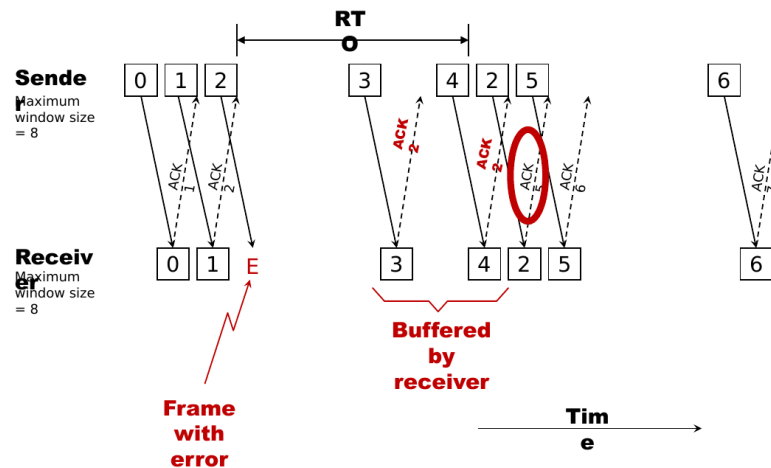
3 Module 4.3 — Selective Repeat, Buffers and Ordered Delivery**3.1 Selective repeat with cumulative ACK**

What the slide says (W4 L3, slide 1)

Idea: the sender should only retransmit dropped/corrupted segments. The receiver **stores** all the correct frames that arrive following the bad one (it needs a **memory buffer** for each sequence number in its receiver window). When the receiver notices a skipped sequence number, it keeps acknowledging the **first in-order sequence number it wants to receive** — this is what we mean by **cumulative ACK**. When the sender times out waiting for an acknowledgement, it **just retransmits the first unacknowledged packet**, not all its successors. Note that the RTO applies independently to each sequence #.

From the lecture slides: W4 L3, slide 2 — Selective Repeat with cumulative ACK (ACK 5 circled)

Selective repeat with cumulative ACK



yellow = buffered out of order; ACK 2 repeated (duplicate ACKs); when 2 arrives, ACK jumps to 5

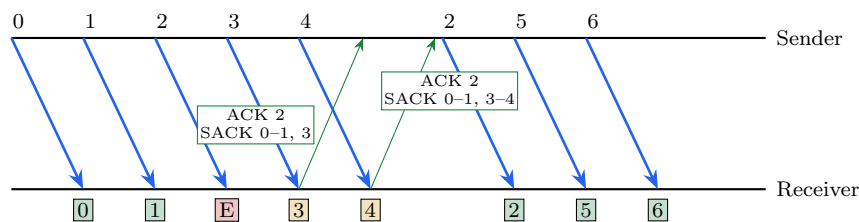
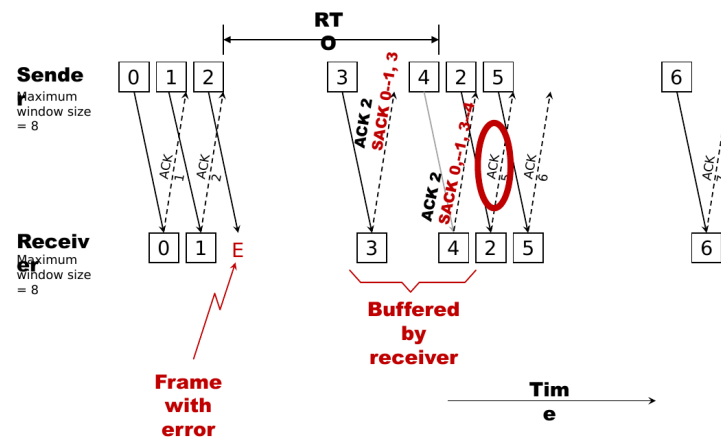
Intuition: Duplicate ACKs as a loss signal

Each out-of-order arrival triggers another “ACK 2”. The sender, seeing repeated ACKs for the same number, gets a strong hint that frame 2 is missing while later frames are getting through. (TCP’s *fast retransmit* resends after 3 duplicate ACKs, without waiting for the RTO — a preview for later weeks.)

3.2 Selective repeat with selective ACK (SACK)

From the lecture slides: W4 L3, slide 3 — Selective Repeat with selective ACK

Selective repeat with selective ACK



the sender learns *exactly* which frames arrived, so it resends only the hole(s)

Intuition: Cumulative vs selective ACK

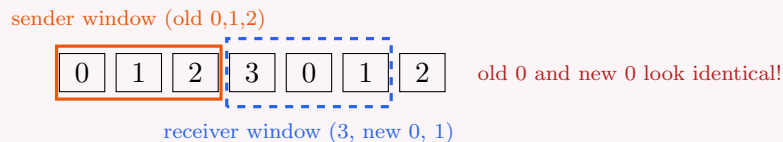
A cumulative ACK says “everything before n arrived” — compact, robust (a lost ACK is covered by the next one), but it can’t describe multiple holes. A SACK also lists the received blocks beyond the hole, so with several losses per window the sender can fix them all in one round trip. TCP’s default is cumulative; SACK is a widely used option (RFC 2018).

3.3 Comparing the three reliability schemes

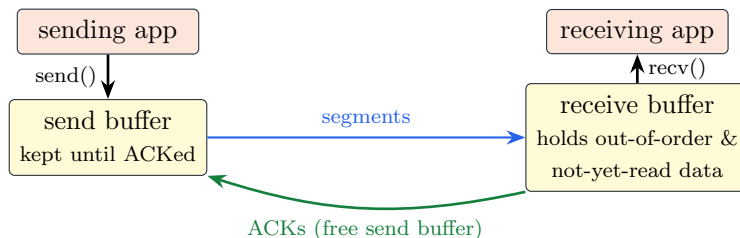
	Stop-and-wait	Go-Back-N	Selective Repeat
Packets in flight	1	up to N	up to N
Receiver buffers out-of-order?	n/a	no (discards)	yes
ACK type	per packet	cumulative	cumulative (+ SACK option)
Timers	1	1 (oldest unACKed)	one per packet
Retransmitted on 1 loss	1	up to N	1
Throughput	$\approx L/\text{RTT}$	$\approx NL/\text{RTT}$ (drops sharply with loss)	$\approx NL/\text{RTT}$
Min. seq # space	2 (1 bit)	$N + 1$	$2N$

Beyond the slides: Why Selective Repeat needs $2N$ sequence numbers

Take 2-bit numbers (0–3) and SR window $N = 3$. Sender sends 0, 1, 2; all arrive, receiver moves its window to 3, 0, 1 — but all three ACKs are lost. The sender times out and resends the *old* 0. The receiver's window contains a 0 too (the *new* one), so it wrongly accepts the old frame as new data. With window $\leq 2^{k-1}$ (here 2), the sender's and receiver's windows can never overlap in number space, and the ambiguity disappears. For Go-Back-N the receiver window is 1, so $N \leq 2^k - 1$ suffices.



3.4 Memory buffers in the transport layer



3.4.1 Receiver-side sockets need memory buffers

What the slide says (W4 L3, slide 5)

Since TCP uses selective repeat, the receiver must **buffer** data that is received out of order — hold packets so that only the “holes” (due to drops) need to be filled in later, without retransmitting packets that were received successfully. Apps read from the receive-side socket buffer when you do a `recv()` call. Even if data is reliably received in order, applications may not always read the data immediately. What if you invoked `recv()` in your socket program infrequently (or never)? For the same reason, **UDP sockets also have buffers**.

3.4.2 Sender-side sockets need memory buffers

What the slide says (W4 L3, slide 6)

The possibility of **packet retransmission** in the future means that data can't be immediately discarded from the sender once transmitted. The transport layer must wait for the ACK of a piece of data before reclaiming the memory for that data.

Formula: Buffer needed to keep the pipe full

The sender must hold all unACKed data, i.e. a full window. To reach full link rate, window $\geq R \times \text{RTT}$, so

$$\text{send buffer} \gtrsim R \times \text{RTT} \quad [\text{bits}]$$

E.g. $1 \text{ Gbps} \times 100 \text{ ms} = 10^8 \text{ bits} = 12.5 \text{ MB}$. This is why OS TCP buffer sizes are tunable (the slide's IBM and Google Cloud links on tuning TCP stack parameters).

3.5 Q2: How much data to keep in flight?

What the slide says (W4 L3, slide 8)

Challenging question! We want to increase throughput. But:

- The **receiving app must keep up**: otherwise the receiver socket buffer will fill up. Once full, subsequent packets are dropped.
- Even if the receiving app is fast, there must be sufficient **buffering for selective repeat**, if some data is dropped/corrupted.
- The **network path** must be able to keep up. We don't want the window to be so large that packets are dropped anyway.

Challenge: the sender must figure out where the bottleneck is: receiving app? socket buffer? a link along the network path? $\Rightarrow$ **Flow control** and **congestion control**.

From the lecture slides: W4 L3, slides 5, 6, 8 — receiver and sender buffers; how much data in flight

Receiver-side sockets need memory buffers

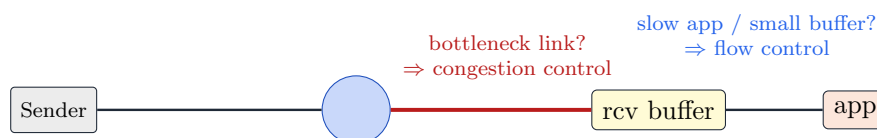
- Since TCP uses selective repeat, the receiver must **buffer** data that is received out of order:
 - e.g., hold packets so that only the "holes" (due to drops) need to be filled in later, without having to retransmit packets that were received successfully
- Apps read from the receive-side socket buffer when you do a `recv()` call.
- Even if data reliably received in order, applications may not **always read the data immediately**
 - What if you invoked `recv()` in your socket program infrequently (or never)?
 - For the same reason, **UDP sockets also have buffers**

Sender-side sockets need memory buffers

- The possibility of **packet retransmission** in the future means that data can't be immediately discarded from the sender once transmitted.
- Transport layer must wait for ACK of a piece of data before reclaiming the memory for that data.

Q2: How much data to keep in flight?

- Challenging question! We want to increase throughput. But:
- The receiving app must keep up: otherwise, **receiver socket buffer will fill up**. Once full, subsequent packets are dropped.
- Even if receiving app is fast, there must be sufficient **buffering for selective repeat**, if some data is dropped/corrupted
- The **network path** must be able to keep up.
- We don't want window to be so large that pkts dropped anyway
- Challenge: The sender must figure out where the bottleneck is: receiving app? socket buffer? A link along the network path?
- Flow control and congestion control



Formula: How big should the window be?

$$\text{window} \leq \min(\text{receiver's advertised window}, \text{congestion window})$$

and ideally $\approx R_{\text{bottleneck}} \times \text{RTT}$: big enough to fill the pipe, small enough not to overflow buffers. The receiver advertises its free buffer in the TCP header's **receive window** field.

3.6 Ordered delivery

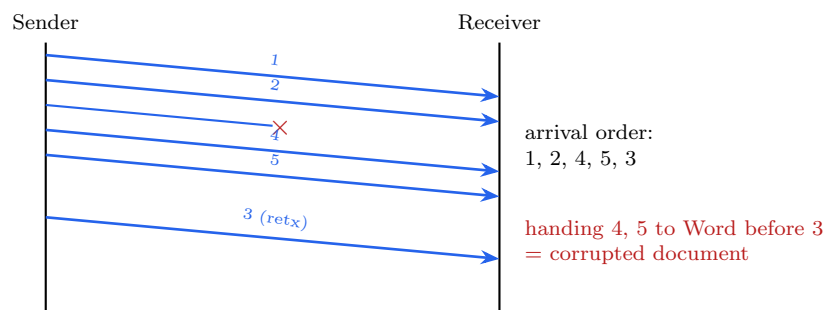
What the slide says (W4 L3, slide 11)

How do apps get performance guarantees? Reliability; **ordered delivery**; resource sharing in the network core — TCP.

3.6.1 Reordering packets at the receiver side

What the slide says (W4 L3, slide 12)

Suppose the receiver gets packets 1, 2 and 4, but not 3 (dropped). Suppose you're downloading a Word document containing a report. What would happen if transport at the receiver directly presented packets 1, 2 and 4 to the Word application?



3.6.2 Why reordering happens, and the fix

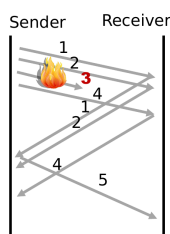
What the slide says (W4 L3, slide 13)

Reordering can happen for a few reasons: **drops** (a retransmission arrives after later packets) and **packets taking different paths** through a network. The receiver needs a general strategy to ensure that data is presented to the application *in the same order that the sender side pushed it*. The receiver uses two mechanisms: **sequence numbers** and the **receiver socket buffer** — we've already seen both for reliability.

From the lecture slides: W4 L3, slides 12–13 — packets 1, 2, 4 but not 3; why reordering happens

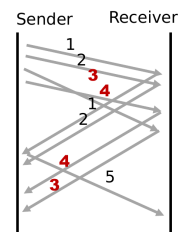
Reordering packets at the receiver side

- Let's suppose receiver gets packets 1, 2, and 4, but not 3 (dropped)
- Suppose you're trying to download a Word document containing a report
- What would happen if transport at the receiver directly presents packets 1, 2, and 4 to the Word application?

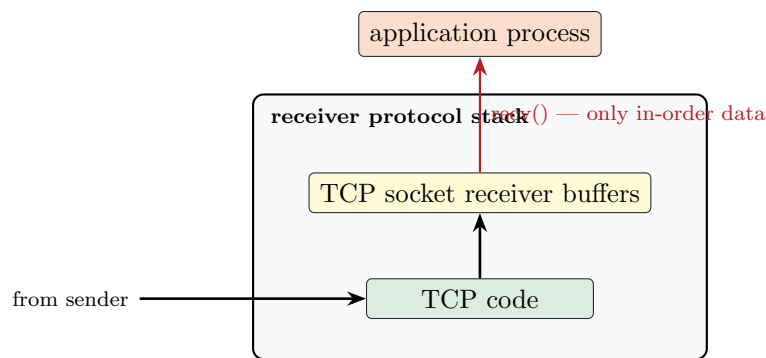


Reordering at the receiver side

- Reordering can happen for a few reasons:
 - Drops
 - Packets taking different paths through a network
- Receiver needs a general strategy to ensure that data is presented to the application *in the same order that the sender side pushed it*
- Receiver uses two mechanisms:
 - Sequence numbers
 - Receiver socket buffer
- We've already seen the use of both of these for reliability



3.6.3 Interaction between apps and TCP: reassembly



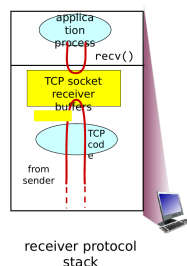
Definition: TCP reassembly

The sender (via the network) deposits data into the receiver socket buffer; an app with a TCP socket reads from that buffer (e.g. `data = sock.recv()`). TCP receiver software only releases data to the application if it is **in order** relative to all other data already read by the application.

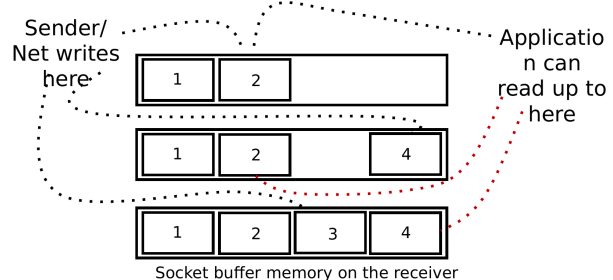
From the lecture slides: W4 L3, slides 14–15 — the receiver protocol stack; TCP reassembly in the socket buffer

Interaction between apps and TCP

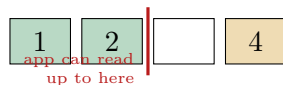
- Sender deposits data in receiver socket buffer
- An app with a TCP socket reads from the TCP receive socket buffer
 - e.g., when you do `data = sock.recv()`
- TCP receiver software only releases this data to the application if the data is **in order** relative to all other data already read by the application
- This process is called **TCP reassembly**



TCP Reassembly



1 and 2 arrive: readable



4 arrives: buffered, not readable



3 arrives: 3 and 4 become readable

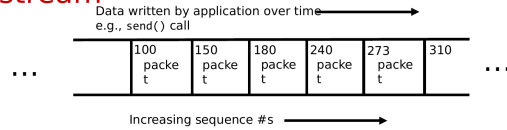
3.6.4 Sequence numbers in the app's stream

What the slide says (W4 L3, slides 16–18)

TCP uses **byte** sequence numbers. Data written by the application over time (e.g. `send()` calls) is carved into packets starting at 100, 150, 180, 240, 273, 310 **Packet boundaries aren't important** for TCP software: TCP is a **stream-oriented** protocol (we use `SOCK_STREAM` when creating sockets). A `recv()` call may return a part of a packet, a full packet, or multiple packets together.

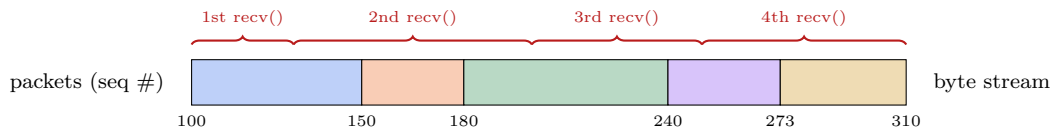
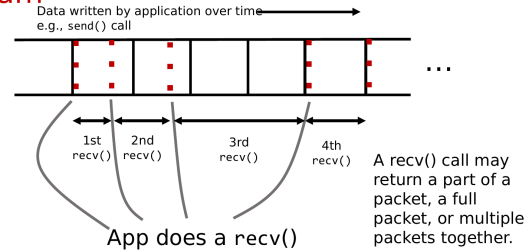
From the lecture slides: W4 L3, slides 17–18 — byte stream; a `recv()` may return part of, one, or several packets

Sequence numbers in the app's stream



Packet boundaries aren't important for TCP software
TCP is a **stream-oriented** protocol
(We use `SOCK_STREAM` when creating sockets)

Sequence numbers in the app's stream



Common pitfall: A classic socket-programming bug (Project P2!)

Never assume one `send()` on one side equals one `recv()` on the other. TCP may merge or split them. Frame your own messages (fixed-length header with a length field, or a delimiter) and loop on `recv()` until you have a full message. UDP, by contrast, preserves message boundaries: one `sendto()` = one datagram = one `recvfrom()`.

3.6.5 Implications of ordered delivery

What the slide says (W4 L3, slide 19)

Packets cannot be delivered to the application if there is an **in-order packet missing** from the receiver's buffer. The receiver can only buffer so much out-of-order data; subsequent out-of-order packets are dropped (it doesn't matter that those packets successfully arrive at the receiver from the sender over the network). **TCP application-level throughput will suffer** if there is too much packet reordering in the network. Data may reach the receiver but won't be delivered to apps upon a `recv()`.

From the lecture slides: W4 L3, slides 19–20 — implications and summary of ordered delivery

Implications of ordered delivery

- Packets cannot be delivered to the application if there is an **in-order packet missing** from the receiver's buffer
 - The receiver can only buffer so much out-of-order data
 - **Subsequent out-of-order packets dropped** (it doesn't matter that those packets successfully arrive at the receiver from the sender over the network)
- **TCP application-level throughput will suffer** if there is too much packet reordering in the network
 - Data may reach the receiver
 - But won't be delivered to apps upon a `recv()`

Summary of TCP ordered delivery

- In-order delivery accomplished through socket buffer and TCP reassembly at receiver
- TCP is a stream-oriented protocol, where the boundaries between packets aren't important
- Significant packet reordering reduces TCP application throughput

Definition: Head-of-line (HOL) blocking

One missing segment blocks delivery of all later segments, even though they have arrived.

Intuition

Think of a queue at a ticket counter where the first person has forgotten their wallet: nobody behind them is served until they return. For a file download this doesn't matter (you need all bytes anyway). For a web page loading 20 images over one TCP connection, one lost packet for image 1 stalls images 2–20 — a major motivation for HTTP/3 over QUIC (UDP), which keeps independent streams so a loss only blocks its own stream.

Summary: TCP ordered delivery (slide)

In-order delivery is accomplished through the socket buffer and TCP reassembly at the receiver. TCP is a stream-oriented protocol, where the boundaries between packets aren't important. Significant packet reordering reduces TCP application throughput.

4 Formula Sheet

Stop-and-wait throughput	$L/(\text{RTT} + L/R) \approx L/\text{RTT}$
Stop-and-wait utilisation	$U = (L/R)/(\text{RTT} + L/R)$
Pipelined throughput	$\min(NL/\text{RTT}, R)$
Window that fills the pipe	$N^* = R \times \text{RTT}/L$ packets ($= R \times \text{RTT}$ bits)
Pipelined utilisation	$U = \min(N(L/R)/(\text{RTT} + L/R), 1)$
TCP next seq #	$x + n$ (seq x , n bytes)
TCP ACK number	next byte expected (cumulative)
Max window, k -bit seq #s	Stop-and-wait: 1 (needs $k = 1$, ABP); GBN: $2^k - 1$; SR: 2^{k-1}
Retransmissions for 1 loss	GBN: up to N ; SR: 1
Send buffer for full rate	$\gtrsim R \times \text{RTT}$
RTO (RFC 6298)	$\text{EstRTT} \leftarrow \frac{7}{8}\text{EstRTT} + \frac{1}{8}\text{Sample}$; $\text{DevRTT} \leftarrow \frac{3}{4}\text{Dev} + \frac{1}{4} \text{Sample} - \text{Est} $; $\text{RTO} = \text{Est} + 4\text{Dev}$

5 Numerical Problems with Worked Solutions

Problem 1. *Stop-and-wait (slide)*

RTT = RTO = 100 ms, packet size 12 Kbit, link rate 12 Mbit/s, no losses. Find the throughput and link utilisation.

Solution

Slide approximation: one packet per RTT $\Rightarrow 12 \times 10^3 / 0.1 = \mathbf{120 \text{ Kbit/s}} \Rightarrow 120\text{K}/12\text{M} = \mathbf{1\%}$.
 Exact: $L/R = 1$ ms, cycle = 101 ms $\Rightarrow$ throughput = $12000/0.101 = 118.8 \text{ Kbit/s}$, $U = 1/101 = 0.99\%$.

Problem 2. *Window to fill the pipe*

For the link above, what window (packets) is needed to reach full link rate? What is the throughput with $N = 20$?

Solution

$N^* = R \times \text{RTT} / L = 12 \times 10^6 \times 0.1 / 12 \times 10^3 = \mathbf{100}$ packets. $N = 20$: $\min(20 \times 12\text{K} / 0.1, 12\text{M}) = \min(2.4, 12) = \mathbf{2.4\text{ Mbps}}$ (20% utilisation).

Problem 3. *Utilisation, 1 Gbps cross-country*

A 1 Gbps link with RTT 30 ms carries 1500-byte packets. Find stop-and-wait utilisation and the window needed for 100%.

Solution

$L/R = 12000/10^9 = 12\text{ }\mu\text{s}$. $U = 0.012/(30 + 0.012) = \mathbf{0.04\%}$. $N^* = 10^9 \times 0.03/12000 = \mathbf{2500}$ packets (= 3.75 MB in flight).

Problem 4. *Pipelined utilisation*

Same as above with a window of $N = 3$ packets: find the utilisation.

Solution

$U = N (L/R) / (\text{RTT} + L/R) = 3 \times 0.012 / 30.012 = \mathbf{0.12\%}$ — tripled, still tiny. Windows must be large on high BDP paths.

Problem 5. *TCP sequence and ACK numbers*

A sender's first byte is 100. It sends segments of 50, 30, 60, 33 and 37 bytes. (a) List each segment's sequence number. (b) List the ACK number after each is received in order. (c) If the second segment is lost and the rest arrive, what ACKs are sent?

Solution

(a) 100, 150, 180, 240, 273 (next would be 310) — the slide's byte boundaries.
 (b) ACK 150, 180, 240, 273, 310.
 (c) After seg 1: ACK 150. Seg 3, 4, 5 arrive but 150–179 is missing: ACK 150, ACK 150, ACK 150 (duplicate ACKs; data buffered). When the retransmitted seg 2 arrives: ACK **310** (everything up to 309 now in order).

Problem 6. *Go-Back-N vs Selective Repeat*

Window 8; frames 0–7 sent; frame 2 is corrupted, all others arrive intact. How many frames cross the network in total (including originals) under (a) GBN, (b) SR?

Solution

(a) GBN: receiver discards 3–7; after the timeout the sender resends 2–7 (6 frames). Total $8 + 6 = \mathbf{14}$. (b) SR: receiver buffers 3–7; only frame 2 is resent. Total $8 + 1 = \mathbf{9}$.

Problem 7. *The slide's Go-Back-N trace*

Using the Go-Back-N slide: frames 0, 1 arrive; frame 2 has an error; 3 and 4 arrive and are discarded; after the RTO, 2, 3, 4, 5, 6 are sent. Which ACKs does the receiver send, and how many frames were wasted?

Solution

ACK 1 (after 0), ACK 2 (after 1); nothing for E, D, D (in the slide's GBN variant the receiver sends no ACK for out-of-order frames); then ACK 3, 4, 5, 6, 7 as 2–6 arrive in order. Wasted transmissions: the corrupted 2 plus the discarded 3 and 4 = **3** frames (3 and 4 had arrived

correctly).

Problem 8. *The slide's Selective Repeat trace*

Frames 0, 1 arrive; 2 has an error; 3, 4 arrive and are buffered; then 2 (retransmitted), 5, 6 are sent. Give the cumulative ACKs, and the SACK information after 3 and after 4.

Solution

ACK 1, ACK 2, (nothing for E), ACK 2, ACK 2 (duplicates), then ACK **5** when 2 arrives (0–4 now in order), ACK 6, ACK 7. With SACK: after 3: “ACK 2, SACK 0–1, 3”; after 4: “ACK 2, SACK 0–1, 3–4”.

Problem 9. *Sequence number space*

(a) With 3-bit sequence numbers, what is the maximum window for GBN and for SR? (b) A protocol needs a window of 64 packets using SR: how many sequence-number bits are needed?

Solution

(a) GBN: $2^3 - 1 = \mathbf{7}$; SR: $2^{3-1} = \mathbf{4}$. (b) $2^{k-1} \geq 64 \Rightarrow k - 1 \geq 6 \Rightarrow k = \mathbf{7}$ bits.

Problem 10. *RTO estimation*

EstimatedRTT = 100 ms, DevRTT = 10 ms. A new SampleRTT = 140 ms arrives. Compute the new EstimatedRTT, DevRTT and RTO ($\alpha = 1/8$, $\beta = 1/4$; update DevRTT using the old EstimatedRTT).

Solution

DevRTT = $0.75 \times 10 + 0.25 \times |140 - 100| = 7.5 + 10 = \mathbf{17.5}$ ms. EstimatedRTT = $0.875 \times 100 + 0.125 \times 140 = 87.5 + 17.5 = \mathbf{105}$ ms. RTO = $105 + 4 \times 17.5 = \mathbf{175}$ ms.

Problem 11. *Buffer sizing*

A 100 Mbps path has RTT 80 ms. What send buffer is needed to keep the pipe full? If the receiver's buffer is only 64 KB, what is the maximum throughput?

Solution

$R \times \text{RTT} = 10^8 \times 0.08 = 8 \times 10^6$ bits = **1 MB**. With a 64 KB window: $64 \times 10^3 \times 8 / 0.08 = \mathbf{6.4 \text{ Mbps}}$ (6.4% of the link) — the receive window is the bottleneck.

Problem 12. *ABP trace*

Using the Alternating Bit Protocol: pkt0 is delivered and ACKed; pkt1 is sent but ACK1 is lost; the sender times out. What does the sender send next, what does the receiver do, and what is ACKed?

Solution

The sender resends pkt1 (same bit). The receiver, now expecting bit 0, sees bit 1 $\Rightarrow$ duplicate: discard the data, re-send **ACK1**. The sender receives ACK1, flips its bit to 0, and sends the next packet as pkt0.

6 Multiple-Choice Questions with Explanations

Q1. The three kinds of guarantees TCP provides are:

- (a) Encryption, compression, routing
- (b) Reliability, ordered delivery, resource sharing
- (c) Addressing, naming, routing
- (d) Low latency, high bandwidth, low jitter

(b).

Q2. The three mechanisms TCP uses to cope with loss are:

- (a) Checksum, CRC, parity
- (b) ACK, RTO, sequence numbers
- (c) NAK, FEC, routing
- (d) Ports, sockets, buffers

(b).

Q3. TCP handles a corrupted packet by:

- (a) Sending a NAK
- (b) Not ACKing it; the sender eventually retransmits
- (c) Correcting it with FEC
- (d) Delivering it anyway

(b). TCP uses only positive ACKs.

Q4. In TCP, who is responsible for retransmitting lost data?

- (a) The receiver
- (b) The routers
- (c) The sender
- (d) The application

(c). "The onus is on the sender."

Q5. Retransmission after an RTO also recovers when:

- (a) Only data is lost
- (b) ACKs are lost or delayed
- (c) The link is down permanently
- (d) The checksum is wrong in the ACK only

(b).

Q6. A good RTO should:

- (a) Be much smaller than RTT
- (b) Predict the RTT (be slightly larger than it)
- (c) Be fixed at 1 s always
- (d) Be set by routers

(b).

Q7. RTT can be measured:

- (a) Only by the receiver
- (b) Directly at the sender
- (c) Only by routers

(d) Only with GPS clocks

(b).

Q8. Sequence numbers are needed primarily because:

- (a) Checksums fail
- (b) Delayed ACKs cause retransmissions that the receiver would mistake for new data
- (c) Routers reorder headers
- (d) Ports run out

(b).

Q9. In stop-and-wait, sequence numbers only need to differ between:

- (a) All packets ever
- (b) Adjacent packets
- (c) Packets and ACKs
- (d) Different connections

(b). Hence 1 bit suffices (ABP).

Q10. The Alternating Bit Protocol uses sequence numbers:

- (a) 0,1,2,3...
- (b) 0 and 1 alternately
- (c) Random
- (d) Byte offsets

(b).

Q11. ABP relies on the channel:

- (a) Never losing packets
- (b) Not reordering packets
- (c) Being encrypted
- (d) Having zero delay

(b).

Q12. With RTT 100 ms, 12 Kbit packets and 12 Mbps link, stop-and-wait achieves about:

- (a) 12 Mbps
- (b) 1.2 Mbps
- (c) 120 Kbps
- (d) 12 Kbps

(c). 1% of link rate.

Q13. Increasing link rate from 12 Mbps to 1.2 Gbps under stop-and-wait (same RTT) makes throughput:

- (a) 100× higher
- (b) Almost unchanged
- (c) Halve
- (d) Zero

(b). Limited by RTT, not link rate.

Q14. TCP sequence numbers count:

- (a) Packets
- (b) Bytes
- (c) ACKs
- (d) Windows

(b).

Q15. A TCP receiver that has received bytes 0–999 sends ACK:

- (a) 999
- (b) 1000
- (c) 0
- (d) 1

(b). Next byte expected.

Q16. The TCP header length field is measured in:

- (a) Bytes
- (b) Bits
- (c) 32-bit words
- (d) Segments

(c). Min 5 (20 bytes).

Q17. “Packets in flight” means:

- (a) Packets in the receiver buffer
- (b) Packets sent but not yet known (by the sender) to be received
- (c) Packets dropped
- (d) Packets queued at the sender

(b).

Q18. With N packets in flight, throughput (below link rate) improves relative to stop-and-wait by a factor of:

- (a) $\sqrt{N}$
- (b) N
- (c) N^2
- (d) $\log N$

(b).

Q19. The window size needed to fill a link equals:

- (a) Link rate / RTT
- (b) Link rate $\times$ RTT
- (c) RTT / packet size
- (d) Packet size $\times$ RTT

(b). In bits; divide by L for packets.

Q20. Sequence numbers wrap around to 0 because:

- (a) Of checksums
- (b) The header field has finite size
- (c) Routers reset them
- (d) Of the receiver window

(b).

Q21. The receiver's window can differ from the sender's because:

- (a) They use different protocols
- (b) The receiver has more recent info about which packets arrived
- (c) The sender is faster
- (d) ACKs are encrypted

(b).

Q22. A packet with a sequence number outside the receiver's window is:

- (a) Buffered
- (b) Dropped
- (c) Delivered immediately
- (d) Forwarded

(b).

Q23. In Go-Back-N, after a missing frame the receiver:

- (a) Buffers later frames
- (b) Discards later frames
- (c) Sends a SACK
- (d) Closes the connection

(b).

Q24. Go-Back-N is wasteful because:

- (a) It needs large receiver buffers
- (b) It retransmits frames the receiver already saw
- (c) It uses NAKs
- (d) It has no timers

(b).

Q25. In Selective Repeat, on timeout the sender retransmits:

- (a) The whole window
- (b) Only the first unacknowledged packet
- (c) Nothing
- (d) All packets ever sent

(b). RTO applies per sequence number.

Q26. A cumulative ACK carries:

- (a) The ranges received
- (b) The first in-order sequence number the receiver wants next

- (c) The last packet received regardless of order
- (d) The window size only

(b).

Q27. A selective ACK (SACK) additionally tells the sender:

- (a) The RTT
- (b) Which ranges beyond the hole were received
- (c) The link rate
- (d) The checksum

(b).

Q28. TCP's default loss-recovery scheme is:

- (a) Go-Back-N
- (b) Selective repeat with cumulative ACK
- (c) Stop-and-wait
- (d) NAK-based

(b).

Q29. With 3-bit sequence numbers, the maximum Selective Repeat window is:

- (a) 3
- (b) 4
- (c) 7
- (d) 8

(b). 2^{k-1} .

Q30. Receiver-side socket buffers are needed because:

- (a) Of encryption
- (b) Out-of-order data must be held and apps may read slowly
- (c) Routers require them
- (d) Of checksums

(b).

Q31. Why can't the sender free data right after transmitting it?

- (a) The app needs it
- (b) It may need to be retransmitted until ACKed
- (c) The OS forbids it
- (d) For the checksum

(b).

Q32. Do UDP sockets have buffers?

- (a) No, never
- (b) Yes, apps may not read immediately
- (c) Only for sending
- (d) Only in IPv6

(b). Slide: “For the same reason, UDP sockets also have buffers.”

Q33. Protecting the receiver’s buffer from being overrun is called:

- (a) Congestion control
- (b) Flow control
- (c) Routing
- (d) Multiplexing

(b). Congestion control protects the network path.

Q34. TCP releases data to the application:

- (a) As soon as any packet arrives
- (b) Only if it is in order relative to data already read
- (c) Only after the connection closes
- (d) In packet-sized chunks only

(b). TCP reassembly.

Q35. A single TCP `recv()` call may return:

- (a) Exactly one packet
- (b) Part of a packet, one packet, or several packets
- (c) Only full application messages
- (d) Only headers

(b). Stream-oriented.

Q36. Heavy packet reordering in the network causes TCP to:

- (a) Increase app throughput
- (b) Reduce app throughput (head-of-line blocking, buffer limits)
- (c) Switch to UDP
- (d) Skip checksums

(b).